

넥스트 레벨 페어스 클론 코딩

6 스크롤을 내렸을 때 나타나는 이미지 - 이론

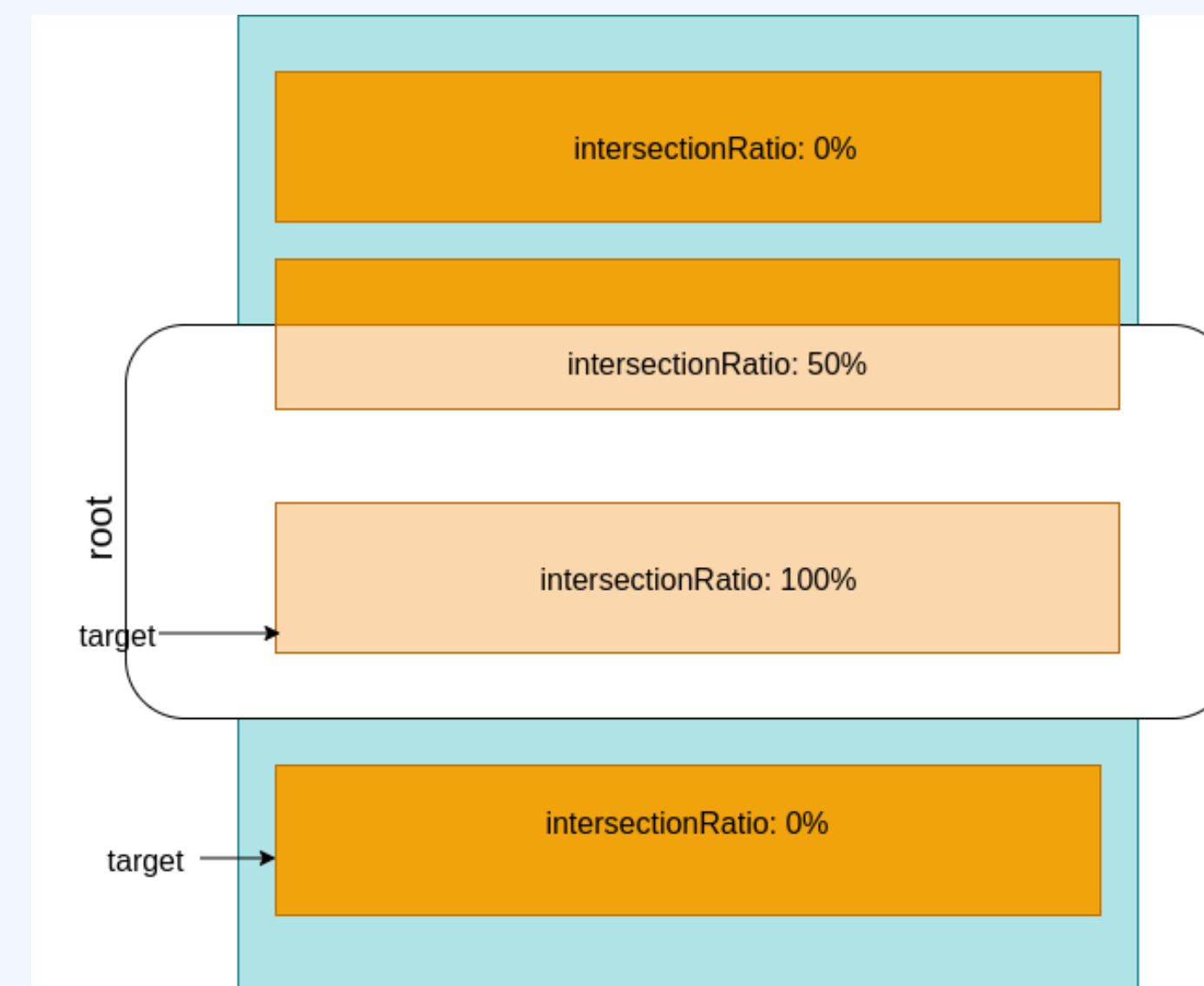
JavaScript IntersectionObserver API

1.

스크롤을 내렸을 때
나타나는 이미지

IntersectionObserver API

- 요소가 유저의 뷰포트에 들어왔는지, 숨겨졌는지 탐지
- 지연 로딩 이미지, 애니메이션 발동, 유저의 활동 추적 등에 용이



<https://misha.agency/javascript/intersection-observer-api.html>

JavaScript IntersectionObserver API - 사용법

2.

스크롤을 내렸을 때
나타나는 이미지

```
let observer = new IntersectionObserver((entries) => {  
  entries.forEach((entry) => {  
    if (entry.isIntersecting) {  
      // 요소가 50% 이상 보일 때 행하는 동작  
    }  
  });  
}, { threshold: 0.5 });  
  
let element = document.querySelector("#my-element");  
observer.observe(element);
```

1. 요소가 화면에 들어오거나 나갈 때 실행될 콜백 함수를 넘겨 IntersectionObserver 객체를 만든다.
2. 관찰하고 싶은 요소를 인자로, IntersectionObserver 객체의 observe 메소드를 호출하여 관찰을 시작한다.
3. 콜백 함수의 단일 인자인 entries(요소의 노출 정도, 위치, 크기 등 포함)로 필요한 정보를 얻는다.

JavaScript IntersectionObserver API - entries

3.

스크롤을 내렸을 때
나타나는 이미지

```
let observer = new IntersectionObserver((entries) => {
  entries.forEach((entry) => {
    entry.boundingClientRect // 관찰된 요소의 뷰포트 내 위치와 크기
    entry.intersectionRatio // 관찰된 요소의 총 교차(화면노출) 비율
    entry.intersectionRect // 뷰포트 내 교차된 영역의 위치와 크기
    entry.isIntersecting // 뷰포트와 현재 교차되고 있는지를 나타내는 boolean
    entry.rootBounds // root 요소의 위치와 크기
    entry.target // 관찰되고 있는 대상 DOM 요소
    entry.time // 교차가 마지막으로 확인된 시간
  });
}, { threshold: 0.5 });
```